LAB 7 – FUZZ TESTING

KOUSHIK C

PES2UG24CS232

42-Salon Booking Application

1. Objective

The aim of this lab was to use fuzz testing to find input-related bugs in the salon booking program. I used the Hypothesis library with pytest to try different inputs and check whether the program could handle them without crashing.

2. Application Overview

The program contains three functions:

- parse_booking() – reads a service name and its duration.
- duration_average() – calculates the average of a list of durations.
- payable_bill() – calculates the bill after adding tax and subtracting the discount.

3. Running the Original Program

I first ran the original application with normal sample inputs. It produced the expected results for all three functions.

Original application output.

```
PS C:\Users\user\Desktop\5th\se\L6_L7_42_salon_booking> python "Lab 7\salon_booking.py"
==== Running parse_booking ====
Enter service:duration in min (e.g. haircut:45): haircut:45
Service and Duration: ('Haircut', 45.0)

==== Running duration_average ====
Enter durations as CSV (e.g. 30,45,60): 30,45,60
Average Duration: 45

==== Running payable_bill ====
Enter bill|discount (e.g. 800|100): 800|100
Payable Bill: 844.0
PS C:\Users\user\Desktop\5th\se\L6_L7_42_salon_booking>
```

4. Fuzz Testing and Bugs Found

I completed the test file using Hypothesis. It generated many different strings and also tested None as an input. The first test run failed for all three functions. Some examples found by Hypothesis were an empty string, a colon by itself, and a pipe by itself. These inputs caused errors because the original functions assumed that the input was always in the correct format.

Initial fuzz testing failures.

```
PS C:\Users\user\Desktop\5th\se\L6_L7_42_salon_booking> python -m pytest "Lab 7/test_salon_booking.py" -v
    | Failed: payable_bill crashed with input None: 'NoneType' object has no attribute 'split'
    | Failing test case: test_payable_bill_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
========================================== short test summary info ==========================================
FAILED Lab 7/test_salon_booking.py::test_parse_booking_no_crash - BaseExceptionGroup: Hypothesis found 3 distinct fail
ures. (3 sub-exceptions)
FAILED Lab 7/test_salon_booking.py::test_duration_average_no_crash - BaseExceptionGroup: Hypothesis found 2 distinct f
ailures. (2 sub-exceptions)
FAILED Lab 7/test_salon_booking.py::test_payable_bill_no_crash - BaseExceptionGroup: Hypothesis found 3 distinct failu
res. (3 sub-exceptions)
========================================== 3 failed in 12.42s ==========================================
PS C:\Users\user\Desktop\5th\se\L6_L7_42_salon_booking>
```

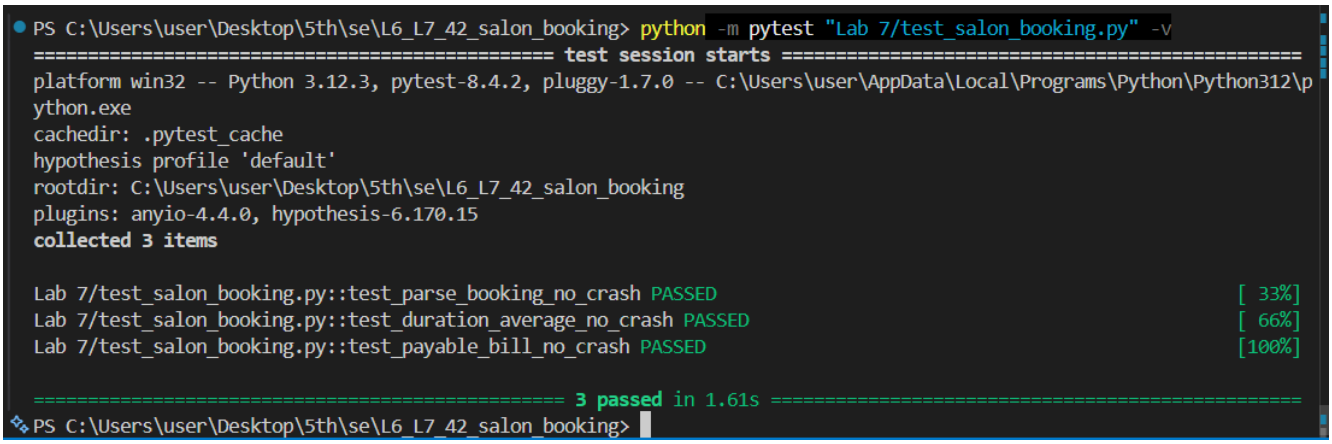
5. Changes Made

I updated salon_booking.py to validate the input before processing it. The program now checks the input type and format, handles invalid numeric values, and rejects negative or non-finite values where appropriate. Invalid input returns None instead of stopping the program with an exception.

6. Testing After the Fix

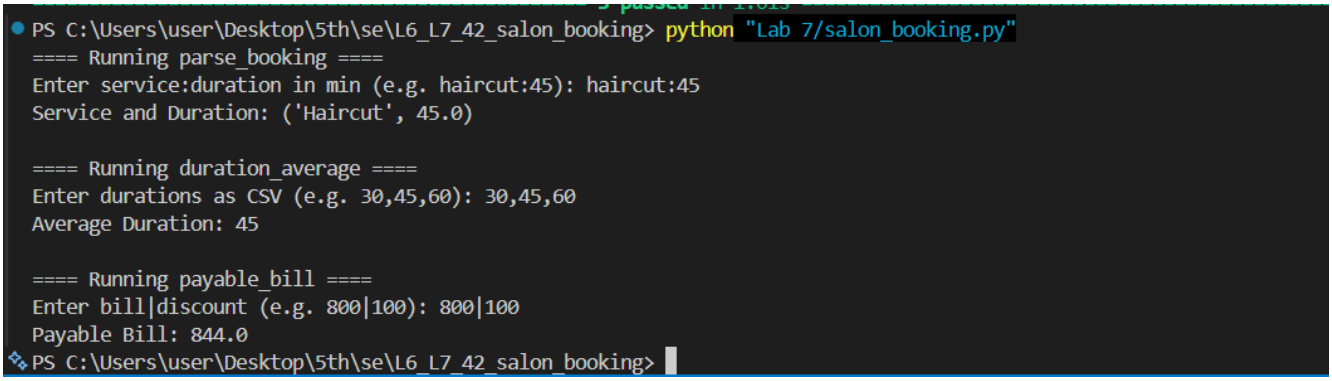
After updating the program, I ran the Hypothesis tests again. All three tests passed: parse_booking, duration_average, and payable_bill. This means the generated inputs used by these no-crash tests did not cause an unhandled exception.

Final pytest output showing 3 passed.



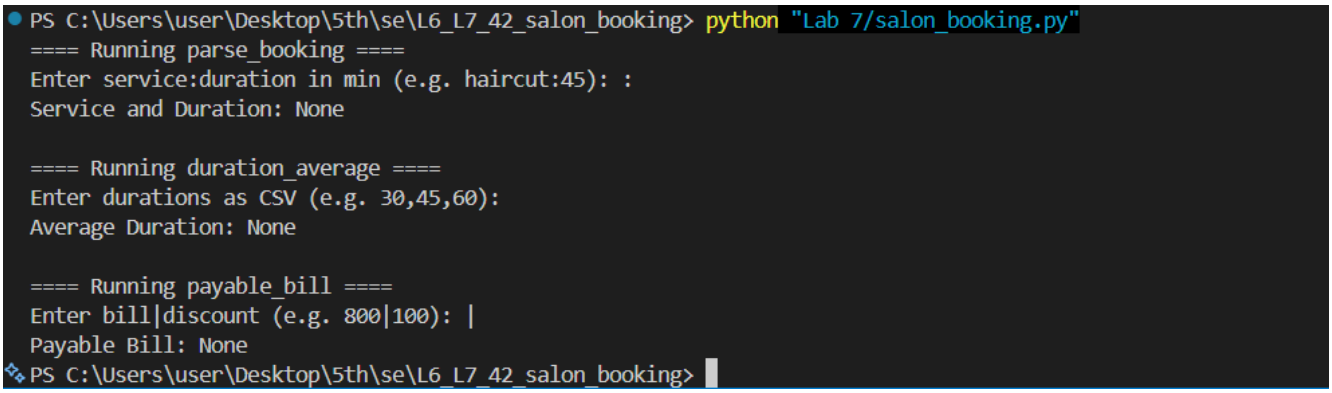
I also ran the fixed application with valid sample inputs. The results remained the same as the original program.

Fixed application with valid inputs.



Finally, I tried invalid inputs manually: ':' for the booking, an empty value for durations, and '|' for the bill. Each function returned None and the application continued without crashing.

Manual invalid-input testing.



7. Conclusion

This lab helped me understand how fuzz testing can reveal input-validation problems that may not appear during normal testing. Hypothesis found inputs that caused the original program to fail. After adding validation and error handling, all three no-crash tests passed, and the application handled the invalid examples safely.